

# Badania operacyjne

## Informatyka 2025/2026

Jan Gawroński, Krzysztof Kopel, Witold Nieć, Jakub Szewczyk

### 1. Wstęp

W ramach naszego projektu skupiliśmy się na optymalizacji planu zajęć na uczelni. W przeciwieństwie do wielu projektów i narzędzi, które skupiają się na optymalizacji od strony studentów (albo grup studentów), my podeszliśmy do problemu od strony nauczycieli. Założyliśmy, że chcemy tworzyć plany, w ramach których nauczyciele pracują tyle, ile wynosi ich pensum, a dodatkowo spełnione są wszystkie inne warunki konieczne do prawidłowego przeprowadzenia zajęć, takie jak chociażby zapewnienie, że zajęcia odbywają się w odpowiedniej sali.

**Ustalenia techniczne:** wykorzystaliśmy język programowania Python 3, bibliotekę programowania liniowego PuLP oraz framework Streamlit do aplikacji graficznej.

### 2. Model matematyczny

#### Ogólna idea:

- Mamy zbiór nauczycieli ( $T$ , teachers), zbiór przedmiotów ( $S$ , subjects), zbiór sal ( $C$ , classrooms) i godzin rozpoczęcia ( $H$ , hour - kolejne liczby naturalne oznaczające „sloty” zajęć, np. 8:00 - 9:30 to 1, 9:45 - 11:15 to 2 itd.).
- Jako dni tygodnia, w których można prowadzić zajęcia, przyjęliśmy okres poniedziałek - piątek:

$$D = \{1, 2, 3, 4, 5\}$$

- Nauczyciele powinni pracować zadaną liczbę godzin w tygodniu (ale potencjalnie różną dla różnych nauczycieli).
- Poszczególni nauczyciele mogą uczyć tylko niektórych przedmiotów.
- Poszczególne sale mogą służyć tylko do uczenia niektórych przedmiotów.
- Nauczyciel może prowadzić w danej godzinie tylko jedno zajęcia.
- W danej sali o jednej godzinie mogą odbywać się tylko jedno zajęcia
- Zajęcia powinny mieć określoną liczbę godzin w tygodniu.

#### Pomocnicze oznaczenia:

1. Dopuszczalne pary przedmiot-nauczyciel:

$$P_{T-S} \subset T \times S$$

2. Dopuszczalne pary przedmiot-sala:

$$P_{S-C} \subset S \times C$$

3. Fakt, że nauczyciel  $t \in T$  prowadzi zajęcia z przedmiotu  $s \in S$  w sali  $c \in C$  o godzinie  $h \in H$  w dniu  $d \in D$  będziemy oznaczali przez:

$$i_{t,s,c,h,d} = 1$$

Jeśli tak nie jest, to:

$$i_{t,s,c,h,d} = 0$$

4. Wymagana liczba godzin zajęć dla danego przedmiotu  $s \in S$ , będziemy oznaczali jako funkcję:

$$r : S \rightarrow \mathbb{N}_+$$

5. Liczba godzin, którą nauczyciel powinien przepracować w ciągu jednego tygodnia (pensum), będziemy oznaczali jako funkcję:

$$p : T \rightarrow \mathbb{N}_+$$

### Metryki i inne funkcje:

1. Całkowita liczba godzin przepracowanych przez nauczyciela w jednym tygodniu to:

$$w_T(t_j, I) = \sum_{s \in S} \sum_{c \in C} \sum_{h \in H} \sum_{d \in D} i_{t_j, s, c, h, d}$$

2. Całkowita liczba dni wolnych, które ma nauczyciel w jednym tygodniu to:

$$f_T(t_j, I) = |D_{t_j, I}|$$

Gdzie zbiór  $D_{t_j, I}$  definiujemy jako:

$$D_{t_j, I} = \left\{ d \mid d \in D, \sum_{s \in S} \sum_{c \in C} \sum_{h \in H} i_{t_j, s, c, h, d} = 0 \right\}$$

### Warunki ograniczające:

Poszczególni nauczyciele mogą uczyć tylko niektórych przedmiotów:

$$\sum_{s \in S} \sum_{c \in C} \sum_{h \in H} \sum_{d \in D} \sum_{t \in T} i_{t, s, c, h, d} = 0, (t, s) \notin P_{T-S}$$

Poszczególne sale mogą służyć tylko do uczenia niektórych przedmiotów:

$$\sum_{s \in S} \sum_{c \in C} \sum_{h \in H} \sum_{d \in D} \sum_{t \in T} i_{t, s, c, h, d} = 0, (s, c) \notin P_{S-C}$$

Nauczyciel może prowadzić w danej godzinie tylko jedno zajęcia:

$$\forall t \in T \forall d \in D \forall h \in H \sum_{s \in S} \sum_{c \in C} i_{t, s, c, h, d} \leq 1$$

W danej sali o jednej godzinie mogą odbywać się tylko jedno zajęcia:

$$\forall c \in C \forall d \in D \forall h \in H \sum_{t \in T} \sum_{s \in S} i_{t, s, c, h, d} \leq 1$$

Dany przedmiot powinien odbywać się określoną liczbę godzin w tygodniu:

$$\forall s \in S \sum_{c \in C} \sum_{h \in H} \sum_{d \in D} \sum_{t \in T} i_{t, s, c, h, d} = r(s)$$

### Zmienna decyzyjna:

Naszą zmienną decyzyjną jest macierz  $I$ , która jest pewnym wartościowaniem zmiennych  $i_{t, s, c, h, d}$  spełniającym warunki ograniczające.

### Funkcja celu:

$$J(I) = \sum_{t \in T} (w_T(t, I) - p(t))^2 + \lambda_1 \sum_{t \in T} f_T(t, I)$$

Parametr  $\lambda_1 \in \mathbb{R}_+$  jest stałą, której wartość dobierzemy eksperymentalnie w czasie tworzenia modelu.

## 4. Opis algorytmów

Przygotowaliśmy łącznie trzy algorytmy, co umożliwiło nam porównywanie ich ze sobą i obserwację rezultatów różnych podejść. Wszystkie nasze algorytmy wykorzystują reprezentację planu zajęć opisaną w poprzednim punkcie. Również generowanie rozwiązania początkowego było prowadzone tak samo we wszystkich algorytmach, poprzez generator opierający się na losowym (ale zgodnym z przyjętymi zasadami) doborze zajęć do planu.

### 4.1 Reprezentacja planu zajęć

W ramach naszego projektu założyliśmy dwa główne sposoby na reprezentowanie danego planu zajęć, czyli głównego obiektu naszej optymalizacji:

- **Macierz 5D:** reprezentacja naturalnie odwzorowująca nasz model matematyczny. Wiemy, że każda zmienna  $i_{t,s,c,h,d}$  może przyjąć wartość 0 albo 1, zatem tworząc macierz 5-wymiarową (w której pierwszy wymiar to wybrany nauczyciel, drugi to przedmiot i tak dalej, zgodnie z przedstawionym modelem matematycznym) możemy przechowywać w niej wszystkie te zmienne jednocześnie. Wówczas plan odczytujemy, patrząc, które ze zmiennych (zmienne na których indeksach) mają wartość 1.
- **Podejście obiektowe:** stworzyliśmy strukturę danych (klasę) w Pythonie, która bardziej naturalnie odwzorowuje pojedynczą lekcję.

```
class Class:
    def __init__(self, teacher: int, subject: int, classroom: int, hour: int, day:
int):
        self.teacher = teacher
        self.subject = subject
        self.classroom = classroom
        self.hour = hour
        self.day = day
```

Lista obiektów tego typu (`list[Class]`) stanowi reprezentację całego planu zajęć. Zaletą tej reprezentacji jest duża czytelność, jak również niższa złożoność obliczeniowa odczytywania wyniku - w przypadku macierzy musimy przejrzeć wszystkie pola (czyli głównie zera, nie mające żadnego wpływu na sam plan), natomiast w przypadku listy przeglądamy tylko faktycznie należące do planu lekcje. W związku z tymi zaletami, to właśnie ta reprezentacja stała się naszą preferowaną opcją.

### 4.2 Algorytm genetyczny

Skupiliśmy się na mutacjach, nie przeprowadzaliśmy krzyżowania. W czasie trwania algorytmu reprezentujemy konfiguracje także w postaci macierzowej, co umożliwia nam szybsze sprawdzanie, czy dana mutacja jest prawidłowa (z punktu widzenia zasad ustalonych w modelu matematycznym).

Użytkownik może zadać zarówno liczbę osobników potomnych, jakie zostaną utworzone w kolejnym pokoleniu, jak i całkowitą liczbę pokoleń.

Mutacja planu polega na wykonaniu następującego algorytmu:

1. Losowo wybieramy jedną lekcję z planu (czyli jedną zmienną  $i_{t,s,c,h,d}$ ).
2. Losowo wybieramy jedną z cech opisujących tę lekcję (nauczyciela, salę, dzień albo godzinę - przedmiotu nie wybieramy, ponieważ zmiana liczby godzin przedmiotu w tygodniu psułaby założony sylabus).
3. Zmieniamy wartość wybranej cechy na inną, jednocześnie dbając o to, aby plan dalej był zgodny z przyjętymi założeniami (czyli jeśli zmieniamy na przykład nauczyciela, to na takiego, który może uczyć danego przedmiotu).

Zatem jedna mutacja polega na bardzo delikatnej zmianie planu, ponieważ modyfikujemy tylko jedną lekcję na raz. Z punktu widzenia macierzy jedna mutacja to po prostu przestawienie jednej ze zmiennych  $i_{t,s,c,h,d}$  z 1 na 0, a pewnej innej zmiennej z 0 na 1.

#### **Pseudokod algorytmu:**

```
if liczba_pokolen == 0:
    return obecne_rozwiazanie

potomkowie = [losowa_mutacja(obecne_rozwiazanie) * liczba_potomkow]

potomkowie = ci potomkowie, którzy są lepsi od obecnego rozwiązania (ale nie więcej niż 3)

wykonaj algorytm rekurencyjnie dla każdego z pozostawionych potomków, z liczbą pokoleń mniejszą o 1 i zwróć najlepsze rozwiązanie
```

### **4.3 Algorytm mrówkowy**

Na początku przygotowujemy dla każdego przedmiotu zbiór wszystkich dopuszczalnych slotów wynikających z ograniczeń dotyczących nauczycieli, sal, dni i godzin. Następnie inicjalizujemy tablicę feromonów jednakową wartością początkową  $\tau_0$ . W każdej iteracji algorytm konstruuje kilka kandydackich rozwiązań, a następnie wybiera spośród nich najlepsze. Budowa pojedynczego rozwiązania przebiega iteracyjnie: dla każdego kolejnego przydziału wyszukujemy aktualnie najbardziej ograniczony przedmiot, czyli taki, dla którego pozostało najmniej poprawnych opcji.

Wybór konkretnego przypisania nie jest całkowicie losowy. Dla każdej dostępnej opcji obliczamy wagę opartą na dwóch składnikach: poziomie feromonu oraz heurystyce. Feromon wzmacnia rozwiązanie podobne do wcześniej dobrych, natomiast heurystyka promuje takie decyzje, które poprawiają zgodność z pensum nauczycieli i ograniczają liczbę ich dni pracy. Następnie wybieramy jedną z opcji metodą ruletki, proporcjonalnie do obliczonych wag. Po wyborze aktualizujemy zajętość nauczyciela i sali, a także zmniejszamy liczbę pozostałych godzin danego przedmiotu.

Jeżeli w którymkolwiek momencie nie da się już poprawnie przypisać kolejnej lekcji, dana próba jest odrzucana. Po zakończeniu generowania kandydatów następuje parowanie feromonów, czyli osłabienie wszystkich ścieżek z użyciem współczynnika  $\rho$ . Następnie na rozwiązaniu najlepszym z danej iteracji odkładany jest dodatkowy feromon, którego wartość zależy od jakości rozwiązania. Dzięki temu algorytm stopniowo faworyzuje konfiguracje prowadzące do niższego kosztu całkowitego.

### **4.4 Integer Linear Programming (ILP)**

Podejście ILP polega na bezpośrednim zapisaniu całego problemu jako zadania optymalizacji liniowej z binarnymi zmiennymi decyzyjnymi. W czasie działania algorytmu dla każdej dopuszczalnej kombinacji nauczyciela, przedmiotu, sali, dnia i godziny tworzymy zmienną oznaczającą, czy dana lekcja odbywa się w tej konfiguracji. W ten sposób plan zajęć jest opisywany nie jako struktura tworzona krok po kroku, ale jako rozwiązanie globalnego układu ograniczeń.

W modelu uwzględniliśmy wszystkie podstawowe warunki wynikające z treści problemu. Każdy przedmiot musi zostać zrealizowany dokładnie w wymaganej liczbie godzin, nauczyciel może prowadzić najwyżej jedno zajęcie w danym terminie, a w jednej sali nie mogą odbywać się równocześnie różne lekcje. Dodatkowo uwzględniamy tylko takie przypisania, które są zgodne z dopuszczalnymi parami nauczyciel-przedmiot oraz przedmiot-sala.

Funkcja celu została zbudowana tak, aby minimalizować odchylenie liczby godzin przepracowanych przez nauczycieli od ich pensum oraz liczbę dni wolnych. Do zapisania tego w postaci liniowej wykorzystaliśmy zmienne pomocnicze opisujące całkowite obciążenie nauczyciela oraz to, czy w

danym dniu prowadzi on zajęcia. Dzięki temu całość dało się sformułować w standardowej postaci akceptowanej przez solver liniowy.

Po zbudowaniu modelu przekazujemy go do biblioteki PuLP. Jeżeli solver znajdzie rozwiązanie optymalne, odczytujemy wartości zmiennych binarnych i na ich podstawie tworzymy listę obiektów `Class`, która stanowi wynik końcowy. Następnie rozwiązanie przechodzi walidację, aby upewnić się, że spełnia wszystkie ograniczenia problemu. W przypadku braku rozwiązania optymalnego algorytm zwraca `None`.

## 4. Aplikacja

W ramach projektu stworzona została aplikacja graficzna umożliwiająca edytowanie danych wejściowych problemu, testowanie poszczególnych algorytmów przy parametrach zadanych w interfejsie oraz porównywanie ich między sobą pod względem czasu działania i historii funkcji kosztu (dla algorytmów iteracyjnych).

### 4.1 Konfiguracja i uruchomienie aplikacji

Aplikacja graficzna wykorzystuje framework Streamlit, który uruchamia aplikację w przeglądarce. Do konfiguracji i uruchomienia programu wymagana jest biblioteka `uv` (menedżer projektów i zależności Pythona).

## System Windows

### Skrypt uruchomieniowy

Aby uruchomić aplikację, wystarczy uruchomić plik `runWindows.sh` (klikając plik dwukrotnie lub wykorzystując wiersz poleceń), znajdujący się w katalogu głównym projektu. Aplikacja uruchomi się w przeglądarce.

### Ręczne uruchomienie

Najpierw należy w wierszu poleceń uruchomić następującą komendę, w celu zsynchronizowania zależności projektu (ważne aby znajdować się w katalogu głównym projektu):

```
C:\Users\User\Desktop\BadaniaOperacyjne>uv sync
```

Następnie aby aplikacja uruchomiła się poprawnie, należy dodać katalog główny projektu do zmiennej środowiskowej `PYTHONPATH`. Można to zrobić między innymi w następujący sposób w wierszu poleceń:

```
>set PYTHONPATH=%PYTHONPATH%;C:\Users\User\Desktop\BadaniaOperacyjne
```

Potem można już uruchomić aplikację:

```
C:\Users\User\Desktop\BadaniaOperacyjne>uv run streamlit run src/app.py
```

## System Linux / Mac OS

### Skrypt uruchomieniowy

Aby uruchomić aplikację wystarczy w terminalu wykonać:

```
~/BadaniaOperacyjne > sh runBash.sh
```

Aplikacja uruchomi się w przeglądarce.

### Ręczne uruchomienie

Aby uruchomić aplikację należy w terminalu uruchomić następującą komendę, w celu zsynchronizowania zależności projektu (ważne aby znajdować się w katalogu głównym projektu):

```
~/BadaniaOperacyjne > uv sync
```

Następnie w celu uruchomienia aplikacji:

```
~/BadaniaOperacyjne > PYTHONPATH=. uv run streamlit run src/app.py
```

Aplikacja powinna uruchomić się automatycznie w przeglądarce.

#### 4.2. Podawanie danych wejściowych problemu

Aplikacja pozwala na wczytanie gotowych przykładowych danych wejściowych lub wprowadzenie własnych danych w zakładce „Dane wejściowe”. W panelu po lewej można również podać wartość ziarna generatora liczb losowych.

Rysunek 1: Widok panelu bocznego i zakładki „Dane wejściowe” interfejsu graficznego

Rysunek 1: Widok panelu bocznego i zakładki „Dane wejściowe” interfejsu graficznego

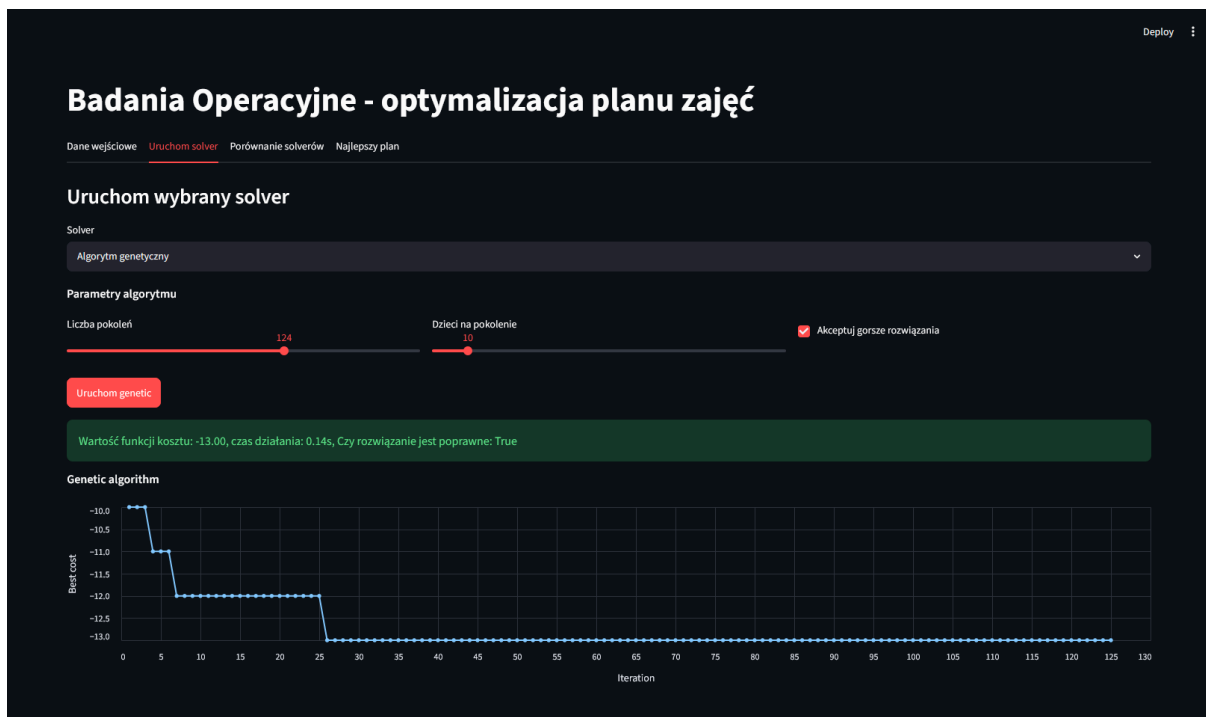
#### 4.3. Pojedyncze uruchomienie algorytmu

W zakładce „Uruchom solver” można wybrać jeden z trzech zaimplementowanych solverów i ustawić parametry wybranego solvera.



Rysunek 2: Widok zakładki „Uruchom solver” - ustawianie parametrów solvera

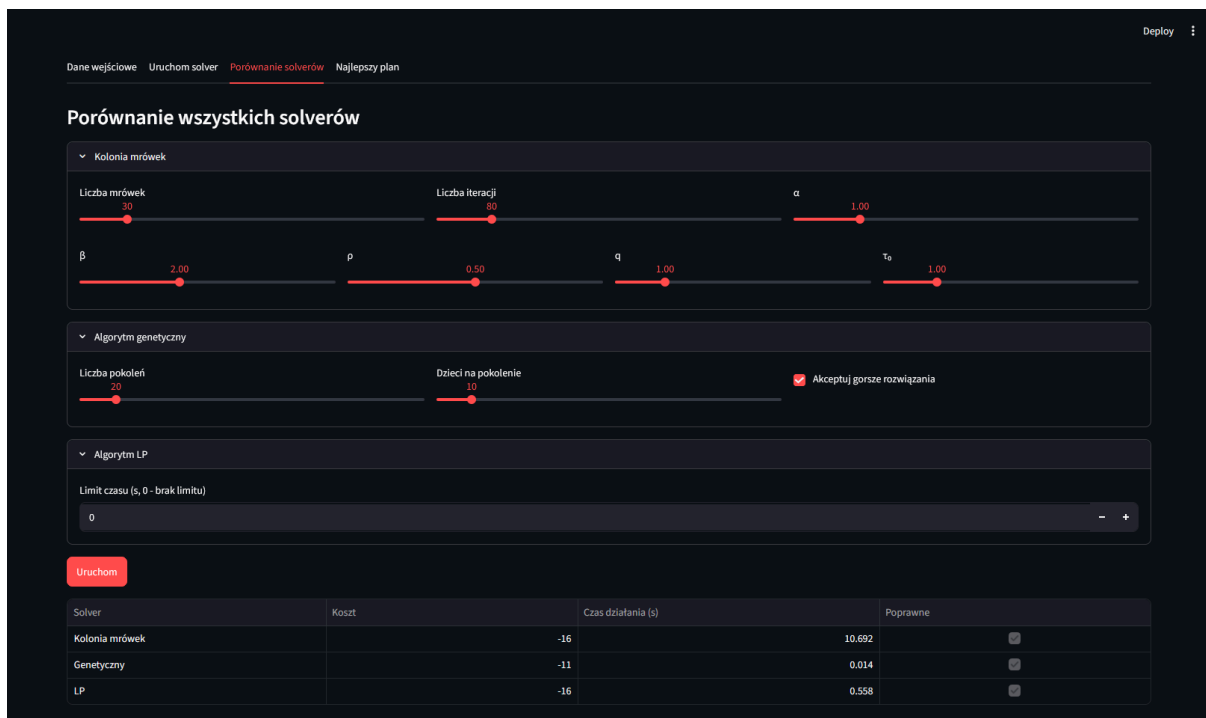
Po uruchomieniu algorytmu ukazują się informacje na temat najlepszej wartości funkcji kosztu, czasie działania algorytmu i poprawności rozwiązania. Dla algorytmów iteracyjnych wyświetlany jest również wykres historii wartości funkcji kosztu dla każdej iteracji.



Rysunek 3: Widok zakładki „Uruchom solver” - wynik algorytmu (algorytm genetyczny)

#### 4.4. Porównanie algorytmów

Zakładka „Porównanie solverów” umożliwia jednoczesne uruchomienie wszystkich solverów w celu ich porównania między sobą.



Rysunek 4: Widok zakładki „Porównaj solvery” - ustawianie parametrów solverów





Rysunek 5: Widok zakładki „Porównaj solvery” - wyniki algorytmów

#### 4.5. Widok najlepszego rozwiązania

W zakładce „Najlepszy plan” można podejrzeć najlepsze dotychczasowe rozwiązanie które zostało dotychczas znalezione podczas pojedynczych uruchomień solverów lub po uruchomieniu porównania solverów.

CONNECTING

## Badania Operacyjne - optymalizacja planu zajęć

Dane wejściowe Uruchom solver Porównanie solverów **Najlepszy plan**

**Najlepszy plan od ostatniego uruchomienia**

Najlepszy algorytm: Kolonia mrówek, najlepszy koszt: -16.00 (czas działania 10.69s)

| Day | Slot | Teacher | Subject | Classroom |
|-----|------|---------|---------|-----------|
| Fri |      | 1 T0    | S2      | R1        |
| Fri |      | 2 T0    | S3      | R2        |
| Fri |      | 3 T0    | S0      | R0        |
| Mon |      | 0 T3    | S1      | R0        |
| Mon |      | 2 T3    | S1      | R1        |
| Mon |      | 3 T3    | S2      | R2        |
| Tue |      | 0 T2    | S3      | R1        |
| Tue |      | 1 T2    | S2      | R2        |
| Tue |      | 3 T2    | S0      | R2        |
| Wed |      | 0 T1    | S3      | R1        |

## 5. Eksperymenty

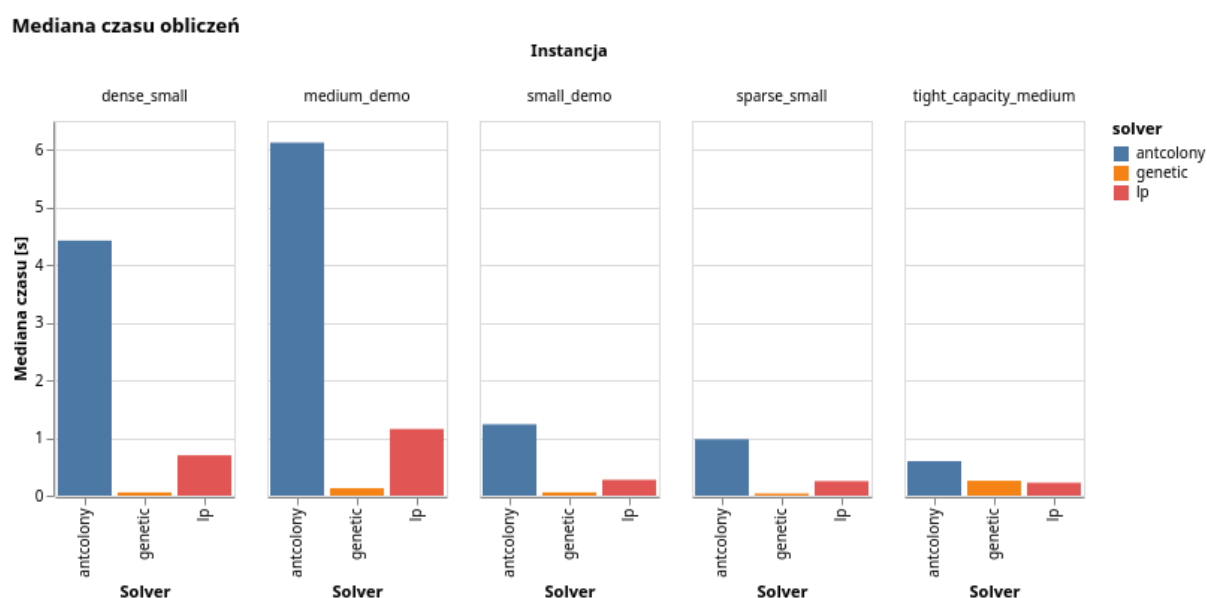
### 5.1 Przypadki rozważanych instancji problemu

W eksperymentach użyto odpowiednio spreparowanych instancji problemu aby możliwie najlepiej pokazać różne cechy rozważanych algorytmów:

- *small\_demo*, *medium\_demo*, *large\_demo* - podstawowe instancje problemu różnej wielkości
- *sparse\_small*, *sparse\_large* - przypadki dla których nauczyciele mogą prowadzić tylko niektóre przedmioty przez co liczba możliwych rozwiązań jest mniejsza
- *dense\_small* - przypadek dla którego każdy nauczyciel może prowadzić każdy przedmiot oraz każdy przedmiot może być prowadzony w każdej sali
- *tight\_capacity\_medium* - przypadek dla którego ciężko znaleźć rozwiązanie (jakikolwiek). Wynika to z licznych ograniczeń harmonogramu oraz krytycznie małej liczby dostępnych okien czasowych.

### 5.2 Wyniki

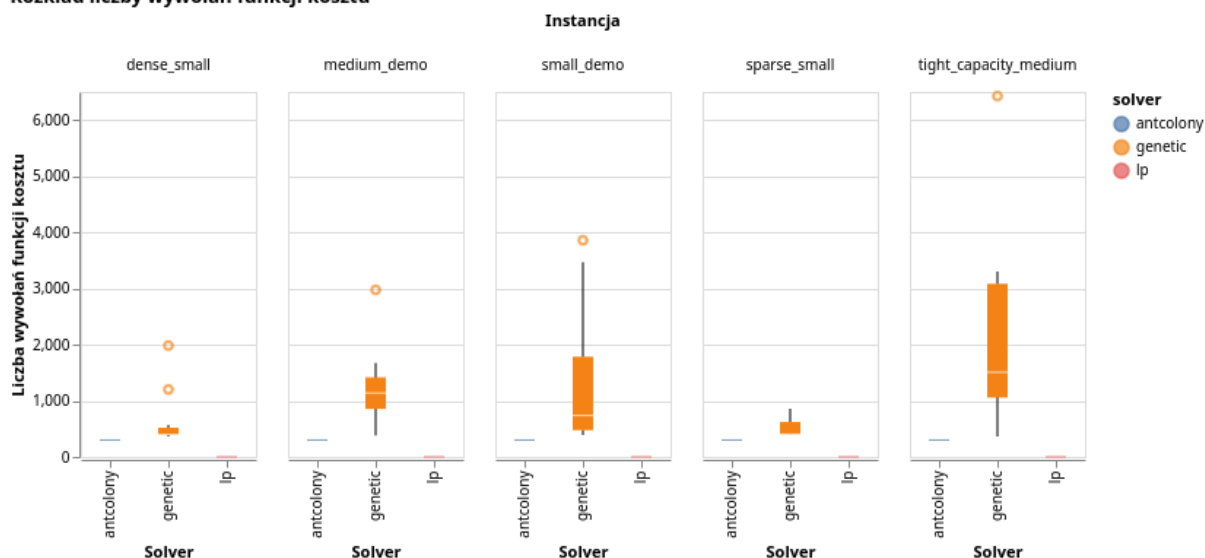
W tym miejscu warto zaznaczyć że kwestia implementacji miała kluczowe znaczenie. Nie należy poniższych wyników interpretować jako porównania algorytmów a poszczególnych implementacji.



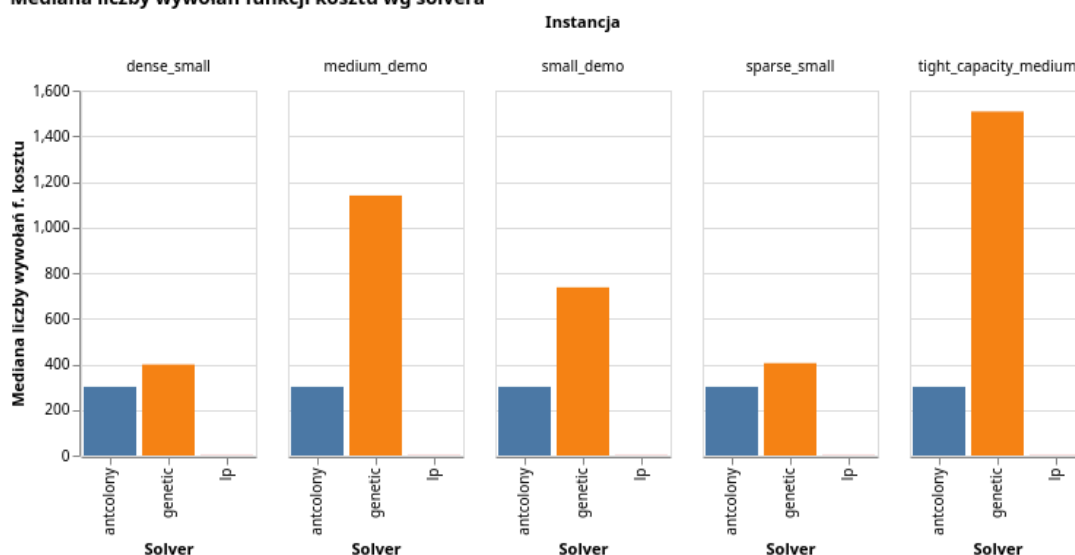
Rysunek 6: Wykres wyników czasowych dla algorytmów

Algorytm genetyczny osiąga tu najlepsze wyniki pod względem czasowym. Nawet dla większych instancji radzi sobie bardzo dobrze. Podejście ILP sprawdziło się raczej stabilnie, dla większości przypadków osiąga kilkukrotnie gorsze wyniki czasowe niż algorytm genetyczny. Wyjątkiem jest przykład *tight\_capacity\_medium*, dla którego znalezienie poprawnego rozwiązania już jest problemem. Podejście algorytmu genetycznego w którym wykonywane są lokalne zamiany (mutacje), dla takiego przypadku optymalizuje znacznie wolniej. Algorytm mrówkowy wymagał zdecydowanie najwięcej czasu obliczeniowego. Wydaje się że w tym przypadku czynnikiem decydującym o wydajności był rozmiar problemu i liczba możliwości.

### Rozkład liczby wywołań funkcji kosztu



### Mediana liczby wywołań funkcji kosztu wg solvera



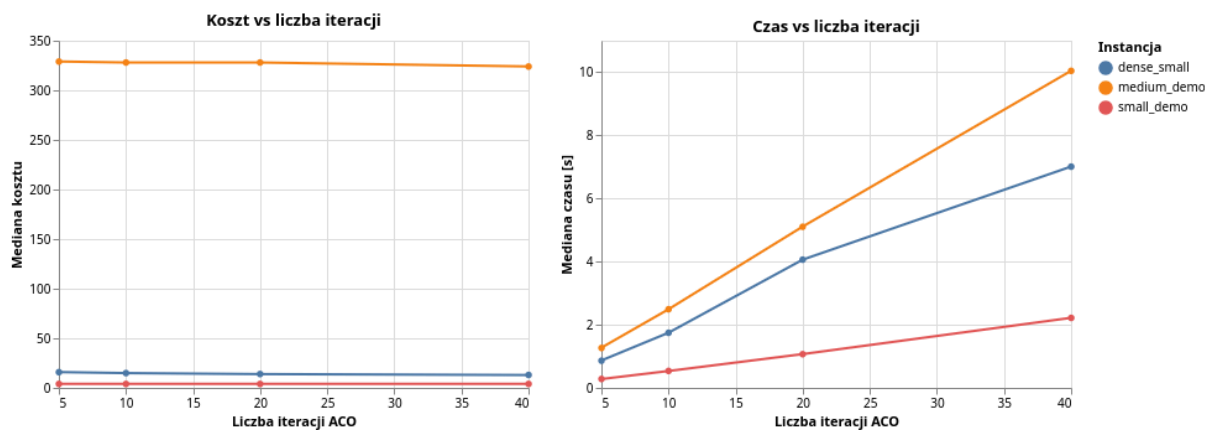
Rysunek 7: Porównanie liczby wywołań funkcji kosztu

W związku z bardzo długim czasem oczekiwania na zakończenie działania algorytmu mrówkowego, ustawiliśmy stałą liczbę mrówek oraz stałą liczbę iteracji - stąd stała wartość wywołań funkcji kosztu. Podejście ILP nie korzysta z funkcji kosztu do optymalizacji dlatego wywoływana jest ona tylko raz - w celu ewaluacji wyniku końcowego. W przypadku algorytmu genetycznego, liczba wywołań funkcji kosztu rośnie wraz ze wzrostem wielkości albo złożoności problemu.

|    | instance_id           | solver    | runs | success_rate | median_cost | mean_cost  | std_cost | min_cost | max_cost | median_time_s | mean_time_s | median_cost_calls | mean_cost_calls |
|----|-----------------------|-----------|------|--------------|-------------|------------|----------|----------|----------|---------------|-------------|-------------------|-----------------|
| 0  | dense_small           | antcolony | 15   | 100.0        | 14.0        | 14.333333  | 1.290994 | 13.0     | 17.0     | 2.962145      | 3.635971    | 301.0             | 286.000000      |
| 1  | dense_small           | genetic   | 10   | 100.0        | 14.0        | 14.200000  | 0.788811 | 13.0     | 15.0     | 0.035118      | 0.054612    | 399.0             | 648.600000      |
| 2  | dense_small           | lp        | 1    | 100.0        | 13.0        | 13.000000  | NaN      | 13.0     | 13.0     | 0.474739      | 0.474739    | 1.0               | 1.000000        |
| 3  | medium_demo           | antcolony | 15   | 100.0        | 328.0       | 327.333333 | 1.877181 | 324.0    | 330.0    | 4.976681      | 5.040554    | 301.0             | 286.000000      |
| 4  | medium_demo           | genetic   | 10   | 100.0        | 326.0       | 325.900000 | 0.994429 | 325.0    | 328.0    | 0.082871      | 0.103102    | 1139.5            | 1258.200000     |
| 5  | medium_demo           | lp        | 1    | 100.0        | 324.0       | 324.000000 | NaN      | 324.0    | 324.0    | 0.973783      | 0.973783    | 1.0               | 1.000000        |
| 6  | small_demo            | antcolony | 15   | 100.0        | 4.0         | 4.133333   | 0.351866 | 4.0      | 5.0      | 0.959336      | 1.073053    | 301.0             | 286.000000      |
| 7  | small_demo            | genetic   | 10   | 100.0        | 4.0         | 4.500000   | 0.707107 | 4.0      | 6.0      | 0.037963      | 0.067980    | 737.0             | 1411.800000     |
| 8  | small_demo            | lp        | 1    | 100.0        | 4.0         | 4.000000   | NaN      | 4.0      | 4.0      | 0.287611      | 0.287611    | 1.0               | 1.000000        |
| 9  | sparse_small          | antcolony | 3    | 100.0        | 26.0        | 26.000000  | 0.000000 | 26.0     | 26.0     | 0.655058      | 0.706083    | 301.0             | 301.000000      |
| 10 | sparse_small          | genetic   | 30   | 100.0        | 26.0        | 26.166667  | 0.461133 | 26.0     | 28.0     | 0.034705      | 0.038720    | 520.5             | 667.666667      |
| 11 | sparse_small          | lp        | 1    | 100.0        | 26.0        | 26.000000  | NaN      | 26.0     | 26.0     | 0.196984      | 0.196984    | 1.0               | 1.000000        |
| 12 | tight_capacity_medium | antcolony | 3    | 100.0        | 24.0        | 24.000000  | 0.000000 | 24.0     | 24.0     | 0.465141      | 0.458512    | 301.0             | 301.000000      |
| 13 | tight_capacity_medium | genetic   | 10   | 100.0        | 24.5        | 24.700000  | 0.823273 | 24.0     | 26.0     | 0.162239      | 0.223418    | 1508.0            | 2228.400000     |
| 14 | tight_capacity_medium | lp        | 1    | 100.0        | 24.0        | 24.000000  | NaN      | 24.0     | 24.0     | 0.179517      | 0.179517    | 1.0               | 1.000000        |

Rysunek 8: Pełne wyniki eksperymentu

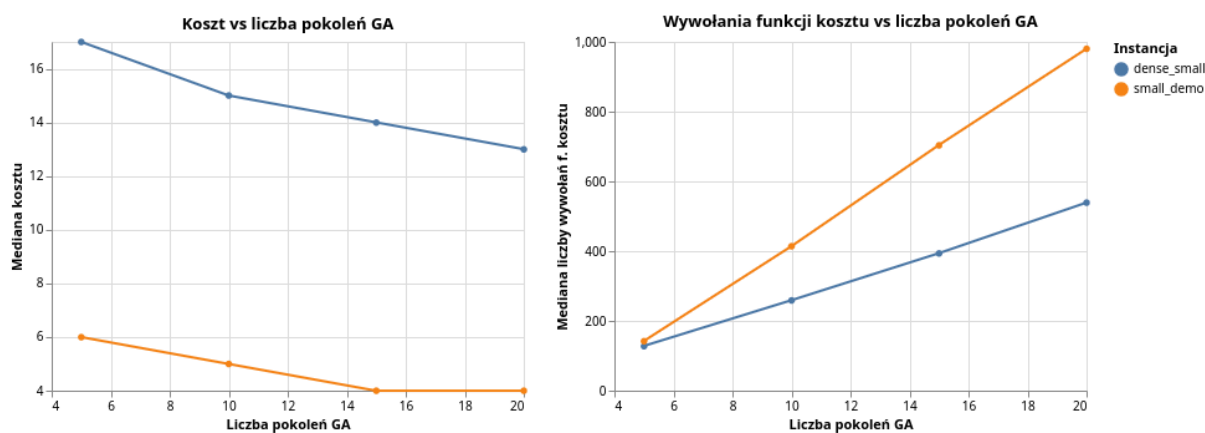
### 5.2.1 AntColony



Rysunek 9: Wpływ liczby iteracji na wynik algorytmu

Wykres zależności kosztu/liczby iteracji pokazuje niemal natychmiastową zbieżność implementacji. Wartość nieznacznie się jednak poprawia przy zwiększaniu iteracji. Niestety ograniczenia czasowe i sprzętowe nie pozwoliły na bardziej rozbudowane eksperymenty.

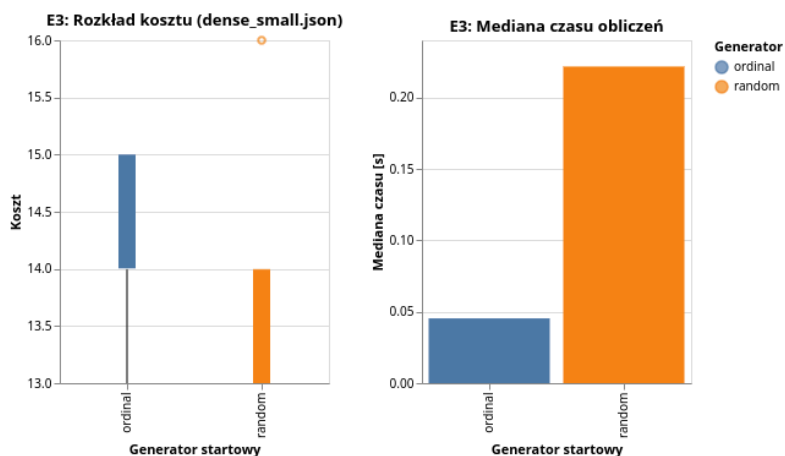
### 5.2.2 Genetic



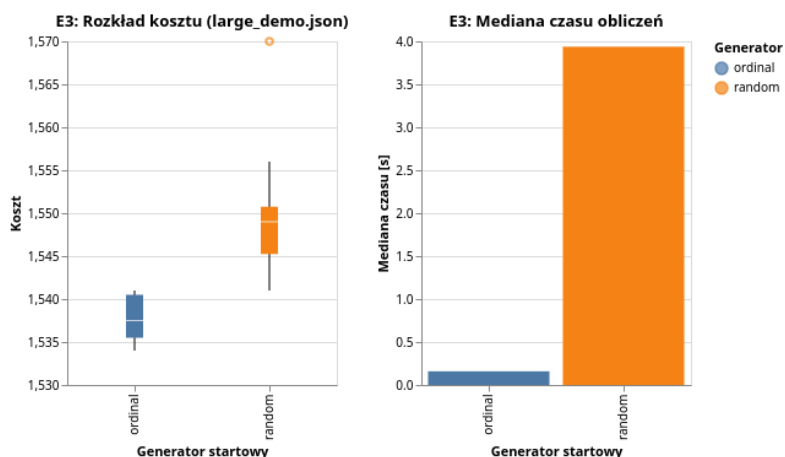
Rysunek 10: Wpływ liczby pokoleń na wynik algorytmu

Dla przykładu *dense\_small* obserwujemy systematyczny spadek funkcji kosztu. Natomiast dla przykładu *small\_demo* wartość ta stabilizuje się szybciej. Zgodnie z oczekiwaniami, liczba wywołań funkcji kosztu stanowi ściśle liniową zależność z liczbą pokoleń.

### 5.3 Porównanie generatorów



Rysunek 11: Porównanie generatorów na przykładzie *dense\_small*



Rysunek 12: Porównanie generatorów na przykładzie *large\_demo*

Ordinal generator wydaje się działać szybciej bez względu na rodzaj czy rozmiar problemu.

## 6. Podsumowanie

W ramach projektu zaimplementowaliśmy algorytmy optymalizujące układanie planu zajęć na uczelni z perspektywy nauczycieli. Stworzyliśmy model matematyczny, który pozwala na znalezienie harmonogramu spełniającego ustalone ograniczenia przy jednoczesnej minimalizacji funkcji kosztu. Do rozwiązania problemu wykorzystaliśmy algorytm genetyczny, algorytm mrówkowy oraz podejście z redukcją do ILP. Na podstawie przeprowadzonych eksperymentów ustaliliśmy, że zdecydowanie najlepsze rezultaty osiągnęła implementacja algorytmu genetycznego.

## 7. Literatura

- Wykłady z przedmiotu „Badania operacyjne”, dr hab. inż. Joanna Kwiecień, prof. AGH

## 8. Dodatek - podział prac

| Element \ student                                                 | Jan Gawroński | Krzysztof Kopel | Witold Nieć | Jakub Szewczyk |
|-------------------------------------------------------------------|---------------|-----------------|-------------|----------------|
| Model matematyczny                                                | X             | X               | X           | X              |
| Struktury danych do reprezentacji problemu i zadawania ograniczeń |               | X               |             |                |
| Generatory rozwiązań początkowych                                 |               |                 | X           | X              |
| Walidator poprawności rozwiązania                                 | X             |                 |             |                |
| Algorytm genetyczny (+opis w dokumentacji)                        |               | X               |             |                |
| Algorytm mrówkowy (+opis w dokumentacji)                          | X             |                 |             |                |
| Redukcja do ILP (+opis w dokumentacji)                            | X             |                 |             |                |
| Aplikacja do prezentacji statystyk                                |               |                 |             | X              |
| Przeprowadzenie eksperymentów                                     |               |                 | X           |                |
| Dokumentacja                                                      |               | X               | X           | X              |